

Technical Handover Guide (HTG)

Project: Password API Assessment Tool

Developer: Sandeep Hari

Language: C# (.NET 8)

Type: Console Application

Overview

This project is a console application that automates authentication and submission to a recruitment API.

It generates password permutations, attempts login using Basic Authentication, and on success builds and uploads a ZIP package containing the project source, assets, and generated dictionary file.

A SAFE MODE is included to allow local testing without making API calls.

Features

- Password permutation generator with character substitutions
- Basic Auth brute-force authentication
- SAFE MODE for local execution
- Recursive ZIP builder with file filtering
- Base64 encoding for API upload
- HTTP client-based submission

Execution Flow

1. Generate password list
2. Save dict.txt file
3. Try authentication with generated passwords
4. Receive upload URL on success
5. Build ZIP (source + assets + dict.txt)
6. Encode ZIP to Base64
7. Upload to API endpoint

Table of Contents

Overview	3
Initial Planning	4
Project Structure	5
Password Permutation Logic.....	6
Authentication Logic	7
Rate Limiting	8
ZIP Packaging	9
Base64 Encoding	10
Upload Process	11
Safe Mode / Local Testing	11
Main Challenges Encountered	12
AI Assistance.....	15
Key Lessons Learned.....	16
Final Thoughts.....	17

Password API Assessment – How Things Got Done (HTG)

Overview

This project was built as part of the Warp Development Password API Assessment using C# and .NET.

The goal of the assessment was to:

- Generate password permutations for the word "password"
- Use those permutations to authenticate against a REST API using Basic Authentication
- Retrieve a temporary upload URL after successful authentication
- Create a ZIP file containing:
 - My CV
 - Source code
 - Generated dictionary file
- Encode the ZIP file as Base64
- Upload the final payload to the provided endpoint

The project was intentionally expanded beyond the minimum requirements to focus on:

- clean structure
- debugging visibility
- safer testing workflows
- proper packaging
- learning and understanding the process instead of only producing output

Initial Planning

Before writing code, I broke the assessment into smaller parts:

1. Generate password permutations
2. Save permutations into dict.txt
3. Authenticate using Basic Auth
4. Manage API rate limiting safely
5. Create ZIP package
6. Convert ZIP into Base64
7. Upload final payload
8. Add local testing workflow (safe mode)
9. Clean and validate project structure

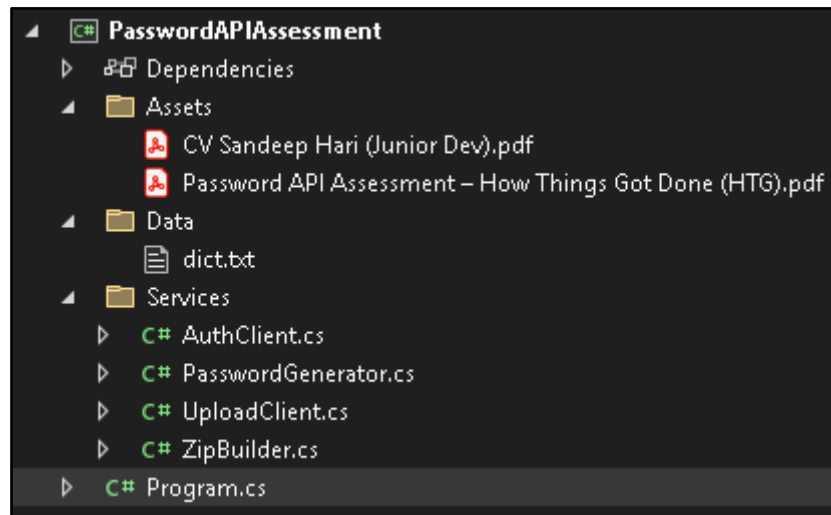
This made the assessment feel much more manageable.

You can view the project history via this GitHub Link:

<https://github.com/SandeepHarii/PasswordAPIAssessment>

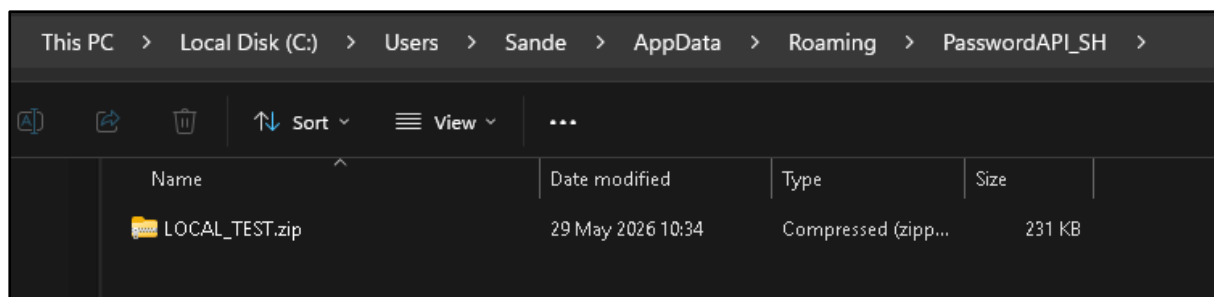
Project Structure

The Final Structure Became:



Generated ZIP files are saved separately in:

AppData/Roaming/PasswordAPI_SH/



This separation prevented accidental recursion and duplicate packaging issues later.

Password Permutation Logic

The first major task was generating all valid password combinations.

The password rules were:

- Any character can be uppercase or lowercase
- a can become @
- s can become 5
- can become 0

Examples:

- password
- Password
- p@55w0rd
- P@ssword

I used recursion/backtracking to generate all combinations.

A `HashSet<string>` was used to avoid duplicates.

The result generated:

1296 passwords

The generated passwords were saved into:

Data/dict.txt

Authentication Logic

Authentication was done using:

Basic Authentication

The API expected:

username: password

encoded as Base64.

Example:

John: password

becomes:

Sm9objpwYXNzd29yZA==

The application:

- loops through each generated password
- builds the Basic Auth header
- sends a GET request
- checks if authentication succeeded

If successful:

the API returns a temporary upload URL

Rate Limiting

The assessment specified:

10 requests per second maximum

To avoid:

- HTTP 429 errors
- temporary blocking

I added:

```
await Task.Delay(120).
```

between requests.

This kept requests safely under the limit.

ZIP Packaging

After successful authentication:

- the project files
- CV
- dictionary

needed to be zipped together.

Initially, packaging caused several issues:

- duplicate files
- nested ZIP files
- locked Visual Studio files
- recursive inclusion of output folders

These problems were solved by:

- excluding:
 - .vs
 - bin
 - obj
 - .git
 - existing .zip files
- separating output folders from source folders
- using controlled directory traversal

The final ZIP includes:

- source code
- Assets folder
- Data folder

without duplicates.

Base64 Encoding

The API required the ZIP bytes to be sent as text inside JSON.

Because raw binary cannot safely exist inside JSON:

- the ZIP bytes were converted into Base64

using:

Convert.ToBase64String(bytes)

This allowed the ZIP to be embedded safely into the JSON payload.

Upload Process

The upload payload contained:

```
{  
  "data": "BASE64_STRING",  
  "name": "Sandeep",  
  "surname": "Hari",  
  "email": "Sandeep.Hari19@gmail.com"  
}
```

The application:

- serialized the payload into JSON
- sent it using HTTP POST
- printed the server response for debugging

Safe Mode / Local Testing

One major improvement added beyond the assessment requirements was:

```
bool isLive = false;
```

This created:

- SAFE MODE
- LIVE MODE

Safe mode:

- skips all API calls
- creates local ZIP files only
- allows debugging without accidental submissions

This was added after realizing repeated testing could:

- trigger upload penalties
- create accidental submissions

This became extremely useful while debugging ZIP issues.

Main Challenges Encountered

1. Relative Path Problems

Initially:

- files saved inside:
 - bin/Debug/net8.0
- instead of project folders

This was solved using:

Path.GetFullPath(...)

and explicit project root resolution.

2. Duplicate Files in ZIP

The ZIP originally contained:

- duplicate CVs
- duplicate dictionary files

Cause:

- files were added manually
- AND included again during directory scanning

Fix:

- use a single-source packaging strategy

3. ZIP Inside ZIP

At one point:

- the generated ZIP file included itself

Cause:

- output folder was inside scanned directories

Fix:

- exclude:
 - output folder
 - .zip files

during traversal.

4. Locked Visual Studio Files

Some .vs indexing files caused:

The process cannot access the file...

Fix:

- exclude .vs
- use safer file sharing modes

AI Assistance

AI tools were used for:

- learning unfamiliar API concepts
- debugging path issues
- understanding ZIP packaging behaviour
- understanding recursion and Base64 encoding
- improving architecture and structure

However:

- the project was heavily iterated on manually
- many debugging decisions required understanding the code and modifying it repeatedly
- several architectural problems were solved through testing and refinement

The focus throughout the project was:

- understanding the logic
- not blindly copying code

Key Lessons Learned

This assessment taught me:

- REST API basics
- Basic Authentication
- Base64 encoding
- recursive algorithms
- ZIP file creation
- filesystem traversal
- debugging path issues
- rate limiting
- safer testing workflows
- separating source/output architecture

Most importantly:

- I learned how quickly project complexity grows when multiple systems interact together.

Final Thoughts

The assessment started off feeling intimidating because it involved:

- APIs
- authentication
- encoding
- ZIP handling
- file traversal

which were all areas I had little experience with beforehand.

Breaking the problem into smaller parts made it manageable.

The debugging and restructuring process ended up teaching me more than simply completing the minimum solution would have.